Malachai Emery

FETCH

Purpose

The ‘fetch’ stage is responsible for giving the next instruction to the pipeline. It also maintains the Program Counter. Finally it calculates the next PC value.

Components

- **Program Counter (PC)**
- **Instruction Memory**
- **Adder (PC + 4)**

Connections

- PC output to Instruction Memory address
- Instruction Memory output to IF/ID pipeline latch
- PC + 4 to IF/ID pipeline latch
- PC update logic receives the new PC

Inputs

- clk
- reset

Outputs

- **Instruction**
- **PC+4**

Outputs to IF/ID latch into Decode stage

- **Instruction**
- **PC+4**

DECODE

Purpose

The ‘decode’ stage uses the instruction and acquires all fields. It then reads register operands from the register file. Takes those operands computes control signals for later stages. Finally it performs sign extension of the immediate field.

Components

- **Control Unit**
- **Register File**
- **Sign Extender**

Connections

- Instruction to Control Unit to control signals
- Instruction rs/rt/rd fields to Register File
- Register file outputs to ID/EX latch
- Immediate field to Sign Extend to ID/EX latch

- Control signals to ID/EX latch

Inputs

- Instruction
- PC+4
- clk
- Write-back data + write register index

Outputs

- Register operand A (rs value)
- Register operand B (rt value)
- Sign-extended immediate
- Control signals (RegDst, ALUSrc, ALUOp, MemRead, MemWrite, MemtoReg, RegWrite, Branch)
- Instruction register fields: rs, rt, rd

Outputs passed through ID/EX latch into Execute stage

- Register operand A
- Register operand B
- Sign-extended immediate
- All control signals
- rs / rt / rd fields

EXECUTE

Purpose

The ‘execute’ stage does all the ALU operations (add, sub, etc.). Then it decides the destination register (rt or rd). Finally it prepares data for the memory stage, and computes the 0 flag for branching.

Components

- **ALU Control Unit**
- **ALU**
- **RegDst multiplexer**
- **ALUSrc multiplexer**

Connections

- ALUOp & funct field to ALU Control to ALU control signals
- Operand A (rs) to ALU input A
- Operand B selected by ALUSrc (rt or immediate) to ALU input B
- ALU output to EX/MEM latch
- Zero flag to EX/MEM latch
- RegDst chooses write register (rt or rd)

Inputs

- Control signals from ID/EX (RegDst, ALUSrc, ALUOp, etc.)
- Register data (A, B)

- Immediate value
- rt, rd fields

Outputs

- **ALU result**
- **Zero flag**
- **Write-data for memory** (operand B)
- **Chosen write-register number**

Outputs passed through EX/MEM latch into Memory stage

- ALU result
- Zero flag
- Write-data
- Write-register
- Memory control signals

MEMORY

Purpose

The ‘memory’ stage carries out the data memory reads and writes for LW/SW instructions. If needed it will determine the necessary branch decision.

Components

- **Data Memory**
- (Optional PCSrc/branch logic if implemented)

Connections

- ALU result → Data Memory address
- Write-data → Data Memory write port
- MEM read/write enables → Data Memory control
- Data Memory output → MEM/WB latch

Inputs

- MemRead, MemWrite control signals
- ALU result (memory address)
- Data-to-write
- Zero flag (if branching done here)

Outputs

- Read data (for LW)
- ALU result passed forward (for WB)
- Write-register number

Outputs passed through MEM/WB latch into Writeback stage

- Read data
- ALU result

- Write-register index
- MemtoReg, RegWrite control signals

WRITEBACK

Purpose

The ‘writeback’ stage determines the final value that will be given to the register. Then it completes the instruction cycle.

Components

- **Writeback MUX** (MemtoReg selects ALU result or memory-read data)

Connections

- ALU result / Memory read data to WB MUX to Register file write port

Inputs

- MemtoReg
- ALU result
- Memory read-data

Outputs

- Write-back data (to register file)
- Write-register index (from MEM/WB latch)

Pipeline

IF to IF/ID to ID

- Instruction
- PC+4

ID to ID/EX to EX

- Register values (rs, rt)
- Immediate
- All control signals
- Register indices

EX to EX/MEM to MEM

- ALU result
- Zero flag
- Write register index
- Write-data for memory
- Memory control signals

MEM to MEM/WB to WB

- Memory read-data
- ALU result
- Write register index

- MemtoReg / RegWrite signals

WB to ID

- Final write-back data goes into the Register File



